

## Chapitre n° 5

### RÉCURSIVITÉ

#### I ÇA MARCHE ÇA ?

On considère la suite  $(u_n)$  définie par récurrence par  $u_0 = 2$  et  $u_n = \frac{1}{2} \left( u_{n-1} + \frac{3}{u_{n-1}} \right)$  qui converge vers  $\sqrt{3}$ . (Au passage démontrer le ça fera plaisir à votre prof de math)

On va créer une fonction en Python pour calculer les termes de cette suite, en suivant directement la définition :

```
1 def u(n):
2     if n == 0 :
3         return 2
4     else :
5         x = u(n-1)
6         return 0.5*(x + 3/x)
```

Et la on se dit : "Mais attendez, on appelle la fonction u dans sa propre définition?" Oui. Comprenons ce qu'il se passe avec l'appel  $u(2)$  :

Pour  $u(2)$  : on est dans le `else`, on arrive à  $x = u(n-1)$ .

L'état de la mémoire est :

$u(2)$  :  $n:2$  et  $x:?$

Le programme fait l'appel  $u(2-1)=u(1)$ , et donc "recommence" l'appel à la fonction u : on est une nouvelle fois dans le `else`, on arrive à  $x = u(n-1)$ .

L'état de la mémoire est :

$u(2)$  :  $n:2$  et  $x:?$   
 $u(1)$  :  $n:1$  et  $x:?$

Le programme fait l'appel  $u(1-1)=u(0)$ , et donc "recommence" l'appel à la fonction u : on est cette dans le `then`, on renvoie 2.

$u(2)$  :  $n:2$  et  $x:?$   
 $u(1)$  :  $n:1$  et  $x:?$   
 $u(0)$  :  $n:0$  et  $\text{return}(2)$

L'appel  $u(0)$  est terminé, on a donc une valeur de x pour  $u(1)$

$u(2)$  :  $n:2$  et  $x:?$   
 $u(1)$  :  $n:1$  et  $x:2$

L'appel  $u(1)$  est terminé, on a donc une valeur de x pour  $u(2)$

$u(2)$  :  $n:2$  et  $x:1.75$

On termine.

**DÉFINITION 1**

Une fonction est dite *récursive* si elle s'appelle elle-même.

Le principe est de définir une fonction  $f$  prenant en argument un entier  $n$  en se ramenant au calcul de  $f(0)$  d'une part de  $f(n)$  en fonction de  $f(n-1)$  :

```
1 def f(n):
2     if n == 0:
3         return ...
4     else :
5         return ... f(n-1) ...
```

C'est bien sûr l'idée de base, modifiable selon les besoins :

- on part de  $n=1$
- on part avec  $n=0$  et  $n=1$
- on calcule avec  $f(n-1)$  et  $f(n-2)$
- etc.

L'important c'est d'avoir une *condition d'arrêt*.

Un autre exemple, sans return :

```
1 def partez(n):
2     if n == 0:
3         print("partez!")
4     else:
5         print(n)
6         partez(n-1)
```

## II ITÉRATIF ET RÉCURSIF

Ce sont (à notre humble niveau) les deux principales façons d'aborder une fonction. Soit avec une boucle, soit en récursif.

Voici deux exemples, classiques parmi les classiques, pour la factorielle :

```
1 def facto_ite(n):
2     """Version iterative """
3     p = 1
4     for k in range(n):
5         p = p *(k+1)
6     return(p)
7
8 def facto_rec(n):
9     """Version recursive """
10    if n == 0:
11        return(1)
12    else:
13        return(n*facto_rec(n-1))
```

**EXERCICE 1.** Parcourir à la main les deux versions pour  $n = 3$ .

**EXERCICE 2.** Écrire une fonction récursive qui donne la valeur de  $u_n$  pour la suite  $(u_n)$  définie par  $u_0 = 1$  et  $u_{n+1} = u_n^2 + 1$ .

### III COMPLEXITÉ ET DANGER

Le coût d'un algorithme récursif est lié au nombre d'appels récursifs. Ce coût peut s'exprimer par une relation de récurrence et n'est pas toujours facile à obtenir.

Par exemple pour la fonction `facto_ite` si on note  $c_n$  le coût pour l'appel `facto_ite(n)` on a On a :

- $c(0) = 1$  car on fait un seul test.
- $c(n) = 2 + c(n-1)$  pour  $n \geq 1$  car on fait un test et une multiplication, et le coût de l'appel  $n-1$ .

On a donc  $c(n) = 2n + 1$ , une complexité linéaire.

**Mais attention :** voici le classique des classiques, Fibonacci.

```
1 def fibonacci(n):
2     if n == 0 or n == 1:
3         return 1
4     else :
5         return fibonacci(n-1) + fibonacci(n-2)
```

**EXERCICE 3.** Observer le code avec quelques valeurs de  $n$  et vous poser des questions.

**EXERCICE 4.** Déterminer le coût de la fonction `fibonacci` et comprendre le problème.

**EXERCICE 5.** Si on modifie le code du début par celui là, quel est le problème ?

```
1 def u(n):
2     if n == 0 :
3         return 2
4     else :
5         return 0.5(u(n-1) + 3/u(n-1))
```

### IV AUTRES EXEMPLES

**EXERCICE 6.** Voici une version récursive de la puissance rapide.

- 1) Parcourir l'appel avec  $n = 8$  puis  $n = 9$ .
- 2) La complexité  $C(n)$  vérifie  $C(0) = 1$  et  $C(n) \leq 4 + C(n//2)$ . Essayer d'évaluer la complexité finale.

```
1 def puissance_rapide(x,n):
2     if n == 0:
3         return 1
4     else:
5         r = puissance_rapide(x,n // 2)
6         if n % 2 == 0:
7             return r * r
8         else :
9             return x * r * r
```

**EXERCICE 7.** Compléter la version récursive de la recherche dichotomique dans un tableau trié

```
1 def recherche_dichotomique(element, liste_triee):
2     if len(liste_triee) == ... :
3         return liste_triee[0] == element
4     m = len(liste_triee) // 2
5     if liste_triee[m] == element :
6         return .....
7     elif liste_triee[m] > element :
8         return recherche_dichotomique(.....)
9     else :
10        return recherche_dichotomique(.....)
```